

Atlas Use Case Library

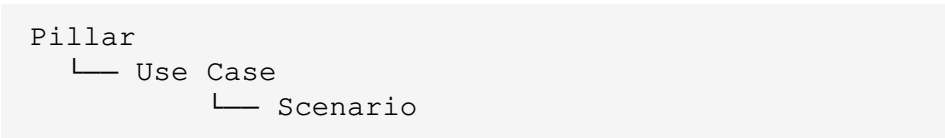
Version 1.0 | Owner: Product Management | Source: Normalized from team artifacts, July–August 2026

Purpose

This document is the authoritative taxonomy for Atlas use case documentation. It establishes a consistent three-level hierarchy — Pillar, Use Case, Scenario — and supersedes the ad hoc naming used across earlier artifacts.

All future use case documents, design specs, demo scenarios, and UX flows should anchor to this taxonomy. Supporting artifacts (conversation scripts, state diagrams, UX flows, agent workflows) are linked from individual use cases but are not first-class entries in the library.

Hierarchy



Pillar — A major value domain. Exists for executive messaging, marketing narratives, and customer mental models. A use case may roll up to more than one pillar.

Use Case — The primary unit of customer value. Answers: *What*

business objective is the customer trying to accomplish? Durable, customer-goal-oriented, independent of UI or AI workflow specifics.

Scenario — A specific situation within a use case. Answers: *What instance of this goal is Atlas supporting?* This is where most detailed artifacts belong.

Pillars

Pillar	Value Domain	Roadmap Maturity
System Intelligence	Know your environment — topology, inventory, relationships, health	GA Dec 2026
Change Intelligence	Change safely — impact analysis, planning, testing, provisioning	GA Dec 2026 (MVP); H1 2027 (full)
Predictive Intelligence	Stay ahead — drift detection, anomaly prediction, DR readiness	H2 2027

Note on "System Understanding" as a use case: System Intelligence is a pillar, not a use case. Customer goals such as Staff Onboarding, Application Discovery, and Audit Preparation are the use cases that *draw from* System Intelligence as their enabling foundation. Listing "System Understanding" as a use case conflates Atlas's internal capability with the customer's objective.

Use Case Index

#	Use Case	Primary Pillar(s)	Status	Primary Persona
UC-01	Vulnerability Remediation —  Spec	System Intelligence, Change Intelligence	Current (GA Dec 2026)	Zach
UC-02	Patch Management —  Spec	Change Intelligence, System Intelligence	Current (GA Dec 2026)	Zach
UC-03	Audit and Compliance —  Spec	System Intelligence	Current (GA Dec 2026)	Derek
UC-04	Staff Onboarding —  Spec	System Intelligence	Current (GA Dec 2026)	Chris
UC-05	Application Discovery and Dependency Analysis —  Spec	System Intelligence	Current (GA Dec 2026)	Angie
UC-06	Change Readiness and Health Assessment —  Spec	System Intelligence, Predictive Intelligence	Current (GA Dec 2026)	Zach
UC-07	Application Change Management —  Spec	Change Intelligence, System Intelligence	Planned (H1 2027)	Kathleen
UC-08	Platform Upgrade and Migration —  Spec	Change Intelligence, System Intelligence	Planned (H1 2027)	Zach

UC-09	Environment Parity and Drift Control —  Spec	Predictive Intelligence, System Intelligence	Planned (H2 2027)	Annette
UC-10	Disaster Recovery Validation —  Spec	Predictive Intelligence, Change Intelligence	Planned (H2 2027)	Greg
UC-11	Capacity Planning and Performance Readiness —  Spec	Predictive Intelligence, Change Intelligence	Planned (H1–H2 2027)	Alex
UC-12	Application Modernization —  Spec	Change Intelligence, System Intelligence	Planned (H1 2027)	Angie
UC-13	Regulatory Change Response —  Spec	System Intelligence, Change Intelligence	Current (GA Dec 2026)	Sage
UC-14	Change Governance and Traceability —  Spec	Change Intelligence	Future Opportunity	Quinn

UC-01: Vulnerability Remediation

Description A security advisory or internal PTF currency check reveals that one or more z/OS components are exposed. The customer needs to assess their exposure, understand what depends on the vulnerable component, and execute a validated remediation — without disrupting

production.

Business Outcome Reduce time from CVE identification to production remediation from days or weeks to hours, with full traceability and no unvalidated changes reaching production.

Current Status: Current — GA Dec 2026

Primary Persona: Zach (z/OS Systems Programmer, experienced)

Secondary Personas: Sage (Security Administrator), Fred (Security Engineer), Alice (z/OS Systems Programmer, mid-level)

Parent Pillar(s): System Intelligence (exposure assessment), Change Intelligence (remediation planning and execution), Predictive Intelligence (proactive risk monitoring)

Scenarios

- **Emergency CVE Response** — External IBM security advisory triggers an immediate exposure check. Atlas identifies affected LPARs, traverses the dependency graph to establish blast radius, and generates a sequenced remediation plan including test and apply steps.
- **Proactive PTF Currency Check** — Routine Atlas check surfaces a FIXCAT security gap before an advisory is issued. Flows into the same remediation path as Emergency CVE Response.
- **Vulnerability Remediation with Active DR Risk** — Remediation is underway on production LPARs, but the DR environment is not yet patched. Atlas flags the open failover exposure and sequences DR remediation into the plan.
- **Multi-LPAR Coordinated Patch Apply** — CVE affects multiple LPARs with interdependencies (e.g., shared Db2 or MQ). Atlas sequences the apply order to avoid compounding downtime risk.

Full Specification: [use-cases/UC-01-vulnerability-remediation-spec.md](#)

Related Existing Artifacts

- `design/flows/uc1-patch-orchestration.md` — detailed conversation script with both reactive and proactive entry points;

maps cleanly as the "Emergency CVE Response" and "Proactive PTF Currency Check" scenarios

- `roadmap.md` — Phase 0 Use Case 4 ("Is my system exposed? What's the blast radius?"); Phase 1 Lean PTF Orchestration
- `Atlas_Demo_Scenarios_v2.md` — Scenario 4 ("Is My System Exposed?") — requires TIB (Aug 19 milestone)

Observations

- Kyle's uc1 is a high-quality scenario script, not a use case definition. It belongs here as the primary reference for the Emergency CVE Response scenario.
 - The proactive PTF check entry point (Step 1B in uc1) is functionally the same use case with a different trigger. It should not be a separate use case.
 - The zSecure Portal integration is a dependency for automated vulnerability PTF detection. If that integration slips, the proactive entry point narrows to what Atlas can derive from PTF inventory alone.
-

UC-02: Patch Management

Description The customer needs to apply PTFs, software patches, or middleware updates to one or more z/OS systems. This is the highest-frequency change type on z/OS and carries meaningful risk due to dependency chains and limited test coverage in most organizations.

Business Outcome Apply patches faster and with higher confidence. Reduce the time spent on manual impact analysis and test planning. Eliminate PTF-related production outages caused by insufficient pre-change validation.

Current Status: Current — GA Dec 2026 (PTF orchestration); H1 2027 (MW/SW patch expansion)

GA Scope Boundaries (Dec 2026):

- Test environments at GA are restricted to **monoplex** configurations (single z/OS image, no Parallel Sysplex). Sysplex test environments are not supported at GA.
- The GA workflow includes a dedicated **Deploy** phase (Application Deployment Engine) between provisioning and test execution. The Application Deployment Engine is a shared Atlas GA capability.
- Test execution at GA is limited to smoke and function tests on monoplex environments. Batch testing and unit testing are out of GA scope.
- ⚠ Cross-product prerequisite-chain resolution is targeted for GA or GA+1 but is **not yet validated**. Do not include in external messaging or customer demos until validated.
- Test-data preparation at GA is a **manual step** performed by the customer before test execution begins.

Primary Persona: Zach (z/OS Systems Programmer, experienced)

Secondary Personas: Alice (mid-level systems programmer), Annette (IT Operations Engineer)

Parent Pillar(s): Change Intelligence (planning, testing, execution), System Intelligence (impact assessment, dependency mapping)

Scenarios

- **Routine Quarterly PTF Maintenance** — Scheduled maintenance window; Atlas generates impact analysis and test plan for a batch of PTFs, provisions isolated monoplex L2 virtual LPAR (customer-operated at GA), deploys application components (Application Deployment Engine), runs smoke/function validation, applies to production in sequence.
- **Emergency Security Patch** — High-priority PTF with short remediation window. Atlas fast-tracks the impact analysis and test cycle. Feeds directly from Vulnerability Remediation (UC-01) when the patch is security-driven.
- **Middleware Software Update (CICS, Db2, MQ)** — Patch scope extends beyond z/OS base to subsystem-level updates. Atlas models cross-subsystem impact and generates a phased rollout.
- **Patch Rollback After Failed Validation** — Test environment or post-apply monitoring reveals an issue. Atlas sequences the

rollback, preserves a known-good state, and documents the failure for future planning.

- **Vendor-Recommended Upgrade** — IBM recommends a specific PTF level for a product upgrade path. Atlas maps the current state to the target level and generates the gap remediation plan.

Full Specification: [use-cases/UC-02-patch-management-spec.md](#)

Related Existing Artifacts

- `roadmap.md` — Phase 1 "Lean PTF Orchestration (Change Intelligence MVP)"; Phase 2 "MW/SW patch orchestration"
- `Atlas_high_level_exec`
`Roadmap_working_Interpretation.md` — GA use case "Patch Orchestration"
- `design/flows/uc1-patch-orchestration.md` — Steps 4–9 describe the patch application flow; the security PTF path is a scenario within this use case
- `Atlas_Demo_Scenarios_v2.md` — Scenario 1 ("What PTFs Are Installed?") is a precursor/discovery step that feeds this use case

Observations

- Patch Management and Vulnerability Remediation share significant execution overlap (impact analysis → test → apply). The distinction is trigger: vulnerability-driven patches enter through UC-01; maintenance-driven patches enter here. The execution path converges.
- Health Check (Kyle's uc9) often surfaces PTF gaps as findings. That makes it a scenario within UC-06 (Change Readiness), not a separate use case — but it feeds Patch Management as a downstream action.

UC-03: Audit and Compliance

Description The customer must produce evidence of configuration

state, access control posture, and change history for an internal or external audit. Typically triggered by a scheduled audit cycle (SOX, PCI, internal) with a fixed deadline.

Business Outcome Reduce audit preparation time from days to hours. Deliver structured, auditor-ready evidence packages from Atlas's continuous environment record without manual assembly.

Current Status: Current — GA Dec 2026

Primary Persona: Derek (Compliance Evidence Provider)

Secondary Personas: Sage (Security Administrator), Zach (z/OS Systems Programmer), Quinn (IT Operations Manager)

Parent Pillar(s): System Intelligence (evidence collection, change history, access analysis), Predictive Intelligence (pattern detection in access and change data)

Scenarios

- **SOX IT General Controls Audit** — Annual audit requiring privileged access reports, configuration compliance against baseline, and 12-month change history. Atlas generates all three artifact types and flags undocumented changes.
- **PCI Compliance Review** — Payment card data environment audit. Atlas identifies all components that touch PCI-scoped data and generates evidence of access controls, encryption posture, and change traceability.
- **Internal Compliance Review** — Internal audit team checks configuration compliance against the organization's own z/OS hardening baseline. Atlas compares current state to defined baseline and quantifies deviation.
- **Undocumented Change Investigation** — Atlas detects changes with no associated change record and surfaces them proactively. Customer investigates before an auditor does.

Full Specification: [use-cases/UC-03-audit-and-compliance-spec.md](#)

Related Existing Artifacts

- `design/flows/uc2-audit-compliance.md` — Full conversation script for SOX scenario; maps as the "SOX IT General Controls Audit" scenario
- `roadmap.md` — Phase 1 "Audit & Compliance artifact generation"
- `Atlas_high_level_exec`
`Roadmap_working_Interpretation.md` — Q3 MVP use case "Audit & Compliance"

Observations

- Regulatory Change Response (UC-13) is a separate use case. Audit prep is about *proving* compliance state; regulatory response is about *remediating* a compliance gap. They share the same Atlas data model but have distinct customer triggers and objectives.
 - The 46 undocumented changes found in Kyle's uc2 (Step 7) are a strong demo hook — this is something Atlas can surface that no existing tool provides.
-

UC-04: Staff Onboarding

Description A new team member — systems programmer, application developer, or operations engineer — needs to understand a z/OS environment they are unfamiliar with. Previously this required weeks of shadowing experienced staff or reading outdated documentation.

Business Outcome Cut time-to-productivity for new z/OS team members from weeks to days. Capture and preserve institutional knowledge before it walks out the door when experienced staff retire.

Current Status: Current — GA Dec 2026

Primary Persona: Chris (z/OS Systems Programmer, early tenure)

Secondary Personas: Alice (mid-level systems programmer), Deb (Application Developer, early tenure), Quinn (IT Operations Manager — the person who needs new staff productive)

Parent Pillar(s): System Intelligence (environment knowledge, topology, change history)

Scenarios

- **New System Programmer** — Atlas orients a new sysprog to the full environment: LPAR topology, subsystem inventory, critical applications, change patterns, and open risk items. Generates a System Brief artifact for reference.
- **New Application Owner** — Atlas orients an application owner to a specific application: program structure, dependencies, data flows, known technical debt, and related change history.
- **New Operations Analyst** — Atlas orients ops staff to operational topology, change window patterns, and current system health state.
- **Knowledge Capture Before Retirement** — A senior team member works with Atlas to document everything they know before leaving. Atlas generates structured handoff artifacts from the live environment rather than from memory.

Full Specification: [use-cases/UC-04-staff-onboarding-spec.md](#)

Related Existing Artifacts

- `design/flows/uc5-staff-onboarding.md` — Full conversation script for New System Programmer scenario; includes the System Brief artifact generation step (Step 4B)
- `roadmap.md` — Phase 1 "Staff Onboarding artifact generation"; customer outcome "I can generate a staff onboarding guide for this environment in under 5 minutes"
- `Atlas_high_level_exec`
`Roadmap_working_Interpretation.md` — Q3 MVP use case "Staff Onboarding"

Observations

- This is one of Atlas's strongest early-value use cases because it does not require Change Intelligence — it lands on System Intelligence alone at GA.
- The "Knowledge Capture Before Retirement" scenario is not currently named in any existing artifact but is explicitly described

in project.md ("expertise walks out the door when a veteran retires"). It deserves its own scenario slot.

UC-05: Application Discovery and Dependency Analysis

Description The customer needs to understand how a specific application is structured, what it depends on, and what would be affected if it were changed. This is a prerequisite for most change work on z/OS and currently requires interrogating multiple siloed tools and people.

Business Outcome Answer in seconds questions that currently require days of research or deep tribal knowledge. Give developers, architects, and system programmers a complete, accurate picture of any application and its dependencies before making a change.

Current Status: Current — GA Dec 2026 (application topology); H1 2027 (deep code-level analysis via ZUnderstand)

Primary Persona: Angie (Application Architect)

Secondary Personas: Kathleen (Application Developer, experienced), Zach (z/OS Systems Programmer), Greg (Infrastructure Architect)

Parent Pillar(s): System Intelligence (topology, dependency map, relationship discovery)

Scenarios

- **Transaction Flow Walkthrough** — User asks Atlas to walk through how a specific business transaction (e.g., a CICS transaction, a REST API call) flows from entry point through middleware to data. Atlas traces the full path across systems.
- **Blast Radius Analysis** — Before making a change, user asks what would be affected. Atlas traverses the dependency graph to identify all downstream components, shared resources, and

potentially impacted applications.

- **Cross-Application Dependency Mapping** — Architect needs to understand how multiple applications share infrastructure or data. Atlas maps the lateral connections between applications and their shared middleware.
- **Application Inventory for a Business Domain** — Management or architecture team wants a complete list of applications supporting a specific business function. Atlas enumerates and maps by CSD group, business service, or naming convention.

Full Specification: [use-cases/UC-05-application-discovery-and-dependency-analysis-spec.md](#)

Related Existing Artifacts

- `architecture/Topology/topology.md` — Defines the three-layer topology model that underpins this use case; this is an engineering/design artifact, not a use case itself
- `Atlas_Demo_Scenarios_v2.md` — Scenario 2 ("What Is My System Running?"), Scenario 6 ("Walk Me Through a CICS Transaction Flow")
- `roadmap.md` — Phase 0 capability "Config-as-Code topology"; Phase 1 capability "Application topology"
- `atlas_unified_use_case_agent_spec.md` — The "Understand" and "Analyze" reusable actions model the agent behavior for this use case

Observations

- `topology.md` is an internal engineering reference document — it describes how Atlas builds the graph, not what customers do with it. It should be linked as a supporting artifact, not treated as a use case or scenario.
 - The "Transaction Flow Walkthrough" scenario is covered by Demo Scenario 6 at the Sept 16 milestone. The dependency on ZUnderstand for deep call chain analysis should be clearly flagged.
-

UC-06: Change Readiness and Health Assessment

Description Before a significant change event — go-live, major upgrade, audit, or peak season — the customer wants to know whether the current system state is sound. This means assessing PTF currency, configuration compliance, security posture, and known risk items as a unified health picture.

Business Outcome Replace manual, multi-tool health checks with a single Atlas-generated assessment. Catch issues before they surface as production incidents. Produce a structured artifact for governance and review.

Current Status: Current — GA Dec 2026

Primary Persona: Zach (z/OS Systems Programmer, experienced)

Secondary Personas: Sage (Security Administrator), Derek (Compliance Evidence Provider), Quinn (IT Operations Manager)

Parent Pillar(s): System Intelligence (configuration state, PTF inventory, cross-middleware analysis), Predictive Intelligence (compounded risk detection)

Scenarios

- **Pre-Go-Live Health Check** — Before a new application or middleware change goes to production, Atlas reviews the full stack for compliance gaps, security findings, PTF currency gaps, and performance constraints. Surfaces issues invisible to single-tool inspection.
- **Pre-Audit Configuration Review** — Generates a health snapshot focused on compliance and access posture. Feeds directly into Audit and Compliance (UC-03).
- **Pre-Change-Window System Review** — Before a planned maintenance window, Atlas confirms the current system state is stable and no unexpected drift has occurred since the last change.
- **Periodic System Health Report** — Scheduled or on-demand

health artifact generation for operational awareness and governance review.

Full Specification: [use-cases/UC-06-change-readiness-and-health-assessment-spec.md](https://github.com/ibm/uc9-health-check/blob/main/use-cases/UC-06-change-readiness-and-health-assessment-spec.md)

Related Existing Artifacts

- `design/flows/uc9-health-check.md` — Full conversation script for the Pre-Go-Live Health Check scenario; the multi-source CICS/Db2/z/OS Connect finding is the canonical example of Atlas's cross-tool advantage
- `roadmap.md` — Phase 1 "Health Check artifact generation"
- `Atlas_high_level_exec`
`Roadmap_working_Interpretation.md` — Q3 MVP use case "Health Check"
- `Atlas_Demo_Scenarios_v2.md` — Scenario 3 ("Do a Health Check on My System Before We Go Live") maps directly to the Pre-Go-Live Health Check scenario

Observations

- Kyle's `uc9-health-check` is a scenario, not a use case. It fits here as the primary script for "Pre-Go-Live Health Check."
 - This use case is a frequent precursor to Patch Management (UC-02) and Audit and Compliance (UC-03). In practice, a health check triggers downstream action in both.
-

UC-07: Application Change Management

Description A developer or systems programmer needs to make a change to an existing application — adding a feature, fixing a bug, updating a COBOL program, modifying a Db2 schema. Atlas provides the impact analysis, test plan, and validation needed to make the change safely.

Business Outcome Let developers make changes on z/OS with

confidence. Replace manual impact analysis and ad hoc testing with Atlas-generated plans and automated validation. Reduce the number of changes that require sysprog involvement.

Current Status: Planned — H1 2027 (limited availability); H2 2027 (developer-native workflows)

Primary Persona: Kathleen (Application Developer, experienced)

Secondary Personas: Deb (Application Developer, early tenure), Zach (z/OS Systems Programmer), Angie (Application Architect)

Parent Pillar(s): Change Intelligence (impact analysis, test generation, validation), System Intelligence (application topology, dependency context)

Scenarios

- **Feature Change with Impact Analysis** — Developer is adding functionality to an existing application. Atlas identifies what else the change touches, generates a test plan, and validates the change in isolation before production.
- **Bug Fix with Regression Testing** — Developer is fixing a defect. Atlas determines blast radius, runs regression tests against the affected application, and confirms no regressions.
- **Db2 Schema Change** — DBA or developer is modifying a database schema. Atlas identifies all applications that use the affected tables and generates an end-to-end test plan.
- **Application Change Initiated by a Developer Without Sysprog Oversight** — (H2 2027) Developer self-serves the full change lifecycle in Atlas, with Atlas acting as guardrails. Sysprog is notified but not required to execute.

Full Specification: [use-cases/UC-07-application-change-management-spec.md](#)

Related Existing Artifacts

- Atlas Use case storyline and product positioning.docx — "Dev Change" is described as one of three core use case patterns; the Atlas flow (Understand → Plan

- Test → Validate → Iterate) applies directly
- `atlas_unified_use_case_agent_spec.md` — "Application Change" cross-use-case mapping: Discover → Analyze → Plan → Execute → Validate → Iterate
- `roadmap.md` — Phase 2 "Application code change support"; Phase 3 "Developer-native Z workflows"

Observations

- This use case is currently future-state for most Atlas customers. The roadmap positions it as limited availability in H1 2027.
 - The "Atlas Use case storyline" doc frames this as one of three canonical change patterns (Dev / Patch / Performance). That framing is useful for executive messaging but does not replace the customer-goal taxonomy here.
-

UC-08: Platform Upgrade and Migration

Description The customer needs to upgrade a core platform component — z/OS version, a major middleware product, or the sysplex configuration. These are the highest-complexity, highest-risk changes in the mainframe calendar and are often attempted only once every several years.

Business Outcome Make major platform upgrades planned, phased, and safe. Eliminate upgrade failures caused by undiscovered compatibility issues. Reduce the time and expertise required to execute a full z/OS or middleware upgrade.

Current Status: Planned — GA Dec 2026 (PTF-level upgrades); H1 2027 (full MW/SW upgrades)

Primary Persona: Zach (z/OS Systems Programmer, experienced)

Secondary Personas: Greg (Infrastructure Architect), Alice (z/OS Systems Programmer, mid-level), Angie (Application Architect)

Parent Pillar(s): Change Intelligence (upgrade planning, sequencing,

testing), System Intelligence (compatibility impact assessment, dependency mapping)

Scenarios

- **z/OS Version Upgrade** — Upgrade from one z/OS version to another (e.g., 2.5 → 3.1). Atlas scopes compatibility issues, sequences the upgrade across LPARs, and runs regression testing at each phase.
- **Middleware Product Upgrade (CICS, Db2, MQ, IMS)** — Major subsystem version upgrade. Atlas models cross-subsystem compatibility, generates the upgrade sequence, and validates each step.
- **Sysplex Reconfiguration** — Changes to sysplex structure, coupling facility definitions, or LPAR assignments. Atlas maps the impact on all systems sharing the sysplex.
- **Phased Migration from Legacy to Modern Stack** — Incremental modernization of platform components over multiple change cycles. Atlas tracks cumulative compatibility state and manages the migration backlog.

Full Specification: [use-cases/UC-08-platform-upgrade-and-migration-spec.md](#)

Related Existing Artifacts

- `design/flows/uc6-zos-upgrade.md` — Full conversation script for z/OS version upgrade; Step 3 (proactive sequencing risk) is a strong Atlas-native capability differentiator
- `roadmap.md` — Phase 2 "MW/SW patch orchestration"
- `Atlas_high_level_exec`
`Roadmap_working_Interpretation.md` — GA use case "z/OS Version Upgrade"

Observations

- Kyle's uc6 is the strongest existing scenario for this use case. The proactive detection of DB2/CICS upgrade sequencing risk (Step 3) demonstrates Predictive Intelligence even within a change workflow.
- "Application Modernization" (UC-12) is a related but distinct use

case — platform upgrade is infrastructure-layer; application modernization is business logic layer. They should not be merged.

UC-09: Environment Parity and Drift Control

Description Production and non-production environments accumulate differences over time — different PTF levels, different configurations, different access controls. This causes test environments to produce misleading results and puts DR environments at risk. The customer needs to detect, quantify, and remediate drift continuously.

Business Outcome Eliminate the "test doesn't look like prod" problem. Catch DR environment drift before a DR test or real event. Maintain configuration parity across environment tiers automatically.

Current Status: Planned — H2 2027

Primary Persona: Annette (IT Operations Engineer)

Secondary Personas: Zach (z/OS Systems Programmer), Alex (Performance Engineer), Greg (Infrastructure Architect)

Parent Pillar(s): Predictive Intelligence (continuous drift monitoring, alerting), System Intelligence (cross-environment comparison, topology diff)

Scenarios

- **Production vs. QA Drift Investigation** — QA cannot reproduce a production issue. Atlas compares the two environments side by side, identifies configuration and PTF differences that explain the behavioral divergence, and generates an alignment plan.
- **Continuous Production Parity Monitoring** — Atlas monitors QA/test environments against production on a schedule and alerts when drift exceeds a threshold.
- **Unauthorized Change Detection** — A configuration change occurs outside a change window with no associated change record. Atlas detects the delta against the last known baseline and

surfaces it for investigation.

- **Post-Change Drift Validation** — After a planned change, Atlas confirms the environment state matches the intended post-change baseline and that no unintended side effects occurred.

Full Specification: [use-cases/UC-09-environment-parity-and-drift-control-spec.md](#)

Related Existing Artifacts

- `design/flows/uc10-environment-drift.md` — Full conversation script for Production vs. QA Drift Investigation; the Db2 ZPARM + PTF mismatch example is a strong concrete scenario
- `roadmap.md` — Phase 3 "Config drift detection"; Phase 1 partial: multi-system comparison
- `atlas_unified_use_case_agent_spec.md` — "Unauthorized Change / Drift" cross-use-case mapping: Detect → Analyze → Decide → Plan → Execute → Validate → Monitor → Close

Observations

- Kyle's uc10 is a scenario script for one specific drift investigation pattern. The broader use case includes proactive/continuous monitoring, which is a separate scenario.
- "Unauthorized Change" as named in the unified agent spec is a scenario within this use case, not a use case in its own right. The customer goal is parity and drift control; unauthorized change detection is one way that goal surfaces.

UC-10: Disaster Recovery Validation

Description The customer needs to verify that their DR environment is production-ready before a scheduled DR test or a real failover event. Current DR validation is manual, infrequent, and often reveals gaps only during the test itself.

Business Outcome Enter every DR test with confidence. Know exactly where the DR environment diverges from production and resolve it before the test, not during. Maintain continuous DR readiness as a documented, auditable state.

Current Status: Planned — H2 2027

Primary Persona: Greg (Infrastructure Architect)

Secondary Personas: Zach (z/OS Systems Programmer), Quinn (IT Operations Manager), Derek (Compliance Evidence Provider — for DR audit evidence)

Parent Pillar(s): Predictive Intelligence (continuous DR parity monitoring, readiness scoring), Change Intelligence (drift remediation), System Intelligence (cross-environment topology diff)

Scenarios

- **Pre-DR-Test Readiness Assessment** — Annual or scheduled DR test is approaching. Atlas compares DR to production, quantifies drift, predicts failure points, and generates a prioritized remediation plan.
- **DR Failover Simulation** — Atlas provisions an isolated environment and simulates a full DR failover under production-level load to validate that the DR environment will sustain operations.
- **Continuous DR Parity Monitoring** — Atlas monitors DR environments daily against production and alerts when any high-severity drift item appears.
- **Post-Incident DR Resync** — After a real failover or a DR test, Atlas identifies what has changed in the DR environment relative to production and generates the resync plan.

Full Specification: [use-cases/UC-10-disaster-recovery-validation-spec.md](#)

Related Existing Artifacts

- `design/flows/uc7-dr-validation.md` — Full conversation script for Pre-DR-Test Readiness Assessment and DR Failover

Simulation; Steps 7–9 cover the simulation and continuous monitoring scenarios

- `roadmap.md` — Phase 3 "Disaster recovery validation"; Phase 3 "Production parity analysis"

Observations

- Kyle's uc7 is the most detailed scenario script in the existing library and maps cleanly onto two of the four scenarios here.
 - DR Validation and Environment Parity (UC-09) share underlying infrastructure (cross-environment comparison, drift detection). DR Validation has a distinct customer trigger (DR readiness, failover confidence) that justifies keeping it as a separate use case.
-

UC-11: Capacity Planning and Performance Readiness

Description The customer needs to understand whether their current system configuration can handle a future load event — peak season, product launch, regulatory deadline — and make proactive configuration changes to prevent performance incidents before they occur.

Business Outcome Prevent performance incidents caused by under-provisioned configuration. Replace reactive capacity management (responding to incidents) with proactive capacity management (predicting and adjusting before events).

Current Status: Planned — H1 2027 (capacity visibility); H2 2027 (full predictive readiness)

Primary Persona: Alex (Performance Engineer)

Secondary Personas: Zach (z/OS Systems Programmer), Annette (IT Operations Engineer), Quinn (IT Operations Manager)

Parent Pillar(s): Predictive Intelligence (load projection, behavioral baseline, proactive alerting), Change Intelligence (configuration change,

test environment provisioning), System Intelligence (current configuration state, capacity inventory)

Scenarios

- **Peak Season Capacity Preparation** — Upcoming high-volume event (Black Friday, month-end, open enrollment). Atlas retrieves historical performance data, projects future load, identifies constraints, and tests configuration changes in isolation before applying to production.
- **Dark Capacity Discovery** — Identifies unused or underutilized capacity across the estate. Surfaces resource reclamation and right-sizing opportunities.
- **Application Performance Diagnosis** — A performance degradation is reported. Atlas traces the symptom through telemetry, transaction topology, and configuration history to identify root cause and generate a remediation plan.
- **Post-Change Performance Regression** — A configuration or application change is suspected to have caused performance degradation. Atlas correlates the change event against performance telemetry to confirm or rule out causation.

Full Specification: [use-cases/UC-11-capacity-planning-and-performance-readiness-spec.md](https://github.com/GoogleCloudPlatform/Atlas/blob/master/design/cases/uc11-capacity-planning-and-performance-readiness-spec.md)

Related Existing Artifacts

- `design/flows/uc4-capacity-planning.md` — Full conversation script for Peak Season Capacity Preparation; the 9-step Black Friday scenario is the canonical example
- `design/flows/uc11-performance-diagnosis.md` — Full conversation script for Application Performance Diagnosis; the Db2 buffer pool root cause example is strong
- `roadmap.md` — Phase 2 "Capacity visibility"; Phase 2 "Application Performance Diagnosis"; Phase 3 "Anomaly prediction (early)"
- `Atlas_high_level_exec`
`Roadmap_working_Interpretation.md` — GA+1 use cases "Application Performance Diagnosis" and "Capacity Planning"

Observations

- Kyle has two separate use case documents (uc4, uc11) that map as two scenarios within one use case. "Capacity Planning" and "Performance Diagnosis" are both instances of the same customer goal: ensuring the system performs correctly under expected load.
 - The Application Performance Diagnosis scenario (uc11) is one of the most compelling Atlas demos because it crosses three pillars within a single conversation.
-

UC-12: Application Modernization

Description The customer has legacy applications on z/OS that need to be updated, refactored, or modernized. They need to understand the current application state, identify what needs to change, and execute the modernization safely — often across applications that no one fully understands anymore.

Business Outcome Make legacy application modernization tractable. Give teams a complete picture of what they are modernizing before they touch it, and a safe path to execute the changes without disrupting production.

Current Status: Planned — H1 2027 (early access); H2 2027 (full support)

Primary Persona: Angie (Application Architect)

Secondary Personas: Kathleen (Application Developer, experienced), Deb (Application Developer, early tenure), Greg (Infrastructure Architect)

Parent Pillar(s): Change Intelligence (remediation planning, phased execution, testing), System Intelligence (technical debt identification, dependency mapping, application topology)

Scenarios

- **Deprecated API Remediation** — An application uses APIs or interfaces that are being sunset in an upcoming platform upgrade. Atlas identifies all affected programs, maps the remediation, and

generates a phased change plan.

- **Monolith Decomposition Planning** — A tightly coupled application with high-change-risk shared code (e.g., a monolithic copybook). Atlas maps the coupling, identifies decomposition boundaries, and sequences a phased decomposition.
- **Application Modernization Assessment** — Architect wants a full picture of modernization candidates: technical debt, deprecated interfaces, unused code, and high-risk coupling patterns. Atlas generates a structured assessment.
- **API Modernization (z/OS to REST)** — Legacy application functionality is being exposed as REST APIs via z/OS Connect. Atlas maps the existing transaction flows and validates the new API endpoints against the underlying implementation.

Full Specification: [use-cases/UC-12-application-modernization-spec.md](#)

Related Existing Artifacts

- `design/flows/uc3-application-modernization.md` — Full conversation script covering Deprecated API Remediation and Monolith Decomposition Planning; Step 7 (ORDCOMM01 decomposition) is the canonical monolith scenario
- `roadmap.md` — Phase 3 "Application modernization support"; Phase 3 "Application modernization topology"
- `Atlas_high_level_exec`
`Roadmap_working_Interpretation.md` — GA+1 use case "Application Modernization"

Observations

- Kyle's uc3 covers two distinct scenarios in one script. Both map cleanly here.
 - This use case has a dependency on ZUnderstand for deep code-level analysis. Application topology from Config-as-Code gives the static picture; ZUnderstand gives the dynamic call chain. Both are needed for full modernization support.
-

UC-13: Regulatory Change Response

Description A new regulatory requirement arrives — data privacy, financial controls, security standards — with a compliance deadline. The customer needs to map the regulation's requirements to their environment, identify gaps, and execute remediation within the deadline.

Business Outcome Turn regulatory change from a scramble into a managed workstream. Know within hours what needs to change, sequence the remediation to avoid operational disruption, and produce a compliance evidence package for regulators.

Current Status: Current — GA Dec 2026

Primary Persona: Sage (Security Administrator)

Secondary Personas: Derek (Compliance Evidence Provider), Lupita (Key Management and Cryptography), Conrad (Application Owner — certificates and secrets), Zach (z/OS Systems Programmer), Quinn (IT Operations Manager)

Parent Pillar(s): System Intelligence (regulated data inventory, access control mapping, gap analysis), Change Intelligence (remediation execution, sequencing), Predictive Intelligence (proactive monitoring for new regulated data)

Scenarios

- **Data Privacy Regulation Implementation** — New data privacy rule requires restricted access, encryption, and audit trails for personal data. Atlas inventories all regulated data, maps access control gaps, and executes the full remediation workstream.
- **Security Controls Mandate** — Regulatory or internal mandate requires specific security configurations (e.g., encryption standards, authentication requirements). Atlas maps current state to required state and generates the gap remediation.
- **Compliance Evidence Package Generation** — After remediation, Atlas generates a structured evidence package documenting current compliance state for regulators.
- **Continuous Regulatory Posture Monitoring** — Atlas monitors

for new data or configurations that fall under regulatory scope (e.g., new datasets containing personal data) and alerts before they become compliance gaps.

Full Specification: [use-cases/UC-13-regulatory-change-response-spec.md](#)

Related Existing Artifacts

- `design/flows/uc8-regulatory-response.md` — Full conversation script for Data Privacy Regulation Implementation; Step 8 (detection of 3 new regulated datasets post-remediation) demonstrates the continuous monitoring scenario
- `roadmap.md` — Phase 1 "Regulatory Change Response"
- `Atlas_high_level_exec`
`Roadmap_working_Interpretation.md` — Q3 MVP use case "Regulatory Change Response"

Observations

- This is distinct from Audit and Compliance (UC-03). Audit prep is about producing evidence of current state; regulatory response is about changing current state to meet a new requirement.
- Lupita (Key Management and Cryptography) and Conrad (Application Owner) are both relevant to the encryption and certificate workstreams within this use case. No existing artifact covers cryptography-specific or certificate-management-specific scenarios — this remains a gap. Conrad and Lupita should be co-primary personas for any future certificate management scenario added to UC-13.

UC-14: Change Governance and Traceability

Description The customer needs every change on z/OS to be traceable to a change record, reviewable in an audit, and attributable to a user and a change window. Today this is largely manual. Atlas should be the system of record for change provenance on z/OS.

Business Outcome Close the gap between what happens on z/OS and what change management systems know about. Give audit teams and operations managers a complete, accurate change history without requiring manual entry.

Current Status: Future Opportunity — H2 2027 (ServiceNow integration); 2028+ (full governance)

Primary Persona: Quinn (IT Operations Manager)

Secondary Personas: Annette (IT Operations Engineer), Derek (Compliance Evidence Provider), Zach (z/OS Systems Programmer)

Parent Pillar(s): Change Intelligence (change attribution, ITSM integration, governance workflow)

Scenarios

- **ServiceNow Change Record Integration** — Every change executed in Atlas is automatically linked to a ServiceNow change record. Atlas acts as the execution layer; ServiceNow remains the system of record for change governance.
- **Undocumented Change Audit Report** — Atlas identifies all changes in a time period that have no associated change record and surfaces them for investigation or retroactive documentation.
- **Change Window Enforcement** — Atlas monitors for changes occurring outside defined change windows and alerts operations management when out-of-window changes are detected.
- **Change Attribution and Rollback History** — Any configuration state can be traced back to the change that caused it, including the user, timestamp, and change record. Rollback is initiated directly from Atlas.

Full Specification: [use-cases/UC-14-change-governance-and-traceability-spec.md](https://github.com/IBM/uc5-staff-onboarding/blob/main/use-cases/UC-14-change-governance-and-traceability-spec.md)

Related Existing Artifacts

- `roadmap.md` — Phase 3 "ServiceNow integration" (bi-directional)
- `design/flows/uc5-staff-onboarding.md` — Step 6 surfaces 46 undocumented changes as a finding; this becomes

the "Undocumented Change Audit Report" scenario

- `design/flows/uc2-audit-compliance.md` — Step 7 shows 46 undocumented changes flagged in a 12-month change history

Observations

- This use case has no dedicated existing artifact. It emerges as a recurring sub-theme in Staff Onboarding, Audit and Compliance, and Environment Drift — always as a finding, never as the primary use case being addressed.
 - The customer goal is real and present in the data. The 46 undocumented changes appearing in two separate Kyle documents underscores it.
 - Full realization requires the ServiceNow integration (H2 2027) and GitHub integration. Partial value is available at GA through Atlas's own change history model.
-
-

Consolidation Recommendations

Duplicate Use Cases — Resolved

Original Artifact Name	Classification	Normalized Destination
kyle uc1: CVE Exposure Assessment and Remediation	Scenario (×2)	UC-01 scenarios: Emergency CVE Response + Proactive PTF Currency Check
kyle uc2: Audit and Compliance	Scenario	UC-03 scenario: SOX IT General Controls Audit
kyle uc3: Application Modernization	Scenarios (×2)	UC-12 scenarios: Deprecated API Remediation + Monolith Decomposition Planning

kyle uc4: Capacity Planning	Scenario	UC-11 scenario: Peak Season Capacity Preparation
kyle uc5: Staff Onboarding	Scenarios (×2)	UC-04 scenarios: New System Programmer + Knowledge Capture Before Retirement
kyle uc6: z/OS Version Upgrade	Scenario	UC-08 scenario: z/OS Version Upgrade
kyle uc7: DR Validation	Scenarios (×2)	UC-10 scenarios: Pre-DR-Test Readiness Assessment + DR Failover Simulation
kyle uc8: Regulatory Change Response	Scenarios (×2)	UC-13 scenarios: Data Privacy Regulation Implementation + Continuous Regulatory Posture Monitoring
kyle uc9: Pre-Production Health Check	Scenario	UC-06 scenario: Pre-Go-Live Health Check
kyle uc10: Environment Drift and Parity	Scenario	UC-09 scenario: Production vs. QA Drift Investigation
kyle uc11: Application Performance Diagnosis	Scenario	UC-11 scenario: Application Performance Diagnosis
Roadmap use case "Health Check"	Scenario	UC-06 scenario: Periodic System Health Report
Roadmap use case "Patch Orchestration"	Use Case	UC-02: Patch Management (retained as use case)

Agent spec "System Intelligence" (as use case)	Pillar	Renamed to System Intelligence pillar; not a use case
Agent spec "Unauthorized Change / Drift"	Scenario	UC-09 scenario: Unauthorized Change Detection
Agent spec "Performance Testing"	Scenario	UC-11 scenario: Post-Change Performance Regression
topology.md	Engineering design artifact	Supporting artifact for UC-05; not a use case or scenario
Atlas Use case storyline.docx	Positioning narrative	Supporting artifact for executive messaging; not a taxonomy input

Naming Conflicts Resolved

Term Used in Existing Artifacts	Normalized Term	Notes
"System Understanding" (as use case)	System Intelligence (pillar)	"System Understanding" is not a customer goal; it is Atlas's enabling capability
"Flow", "Conversation", "Journey", "Interaction"	Supporting artifact	These describe how Atlas works, not what the customer is trying to accomplish
"Drift" (as standalone use case)	Scenario within UC-09	Drift detection is one scenario within the broader Environment Parity use case
"Health Check" (as standalone use case)	Scenario within UC-06	Health Check is a specific instance of the Change Readiness goal

Gap Analysis

Close-In Gaps (adjacent to current roadmap)

Gap	Evidence	Recommended Action
Storage Management	Karla and Elsa exist as personas with TBD use case themes; no storage-related use case exists anywhere in the artifact set	Add UC-15: Storage Management and Optimization. Scenarios: DASD capacity planning, storage group configuration review, volume allocation health check. Primary persona: Karla.
Key Management, Cryptography, and Certificates	Lupita (Key Management) and Conrad (Application Owner — certificates and secrets) both exist as personas; no use case covers key rotation, certificate lifecycle management, or cryptographic compliance in depth	Add scenarios to UC-13 (Regulatory Change Response) for cryptography and certificate-management regulatory requirements. Co-primary personas: Lupita and Conrad. Longer term may warrant a standalone use case at H2 2027+.

AI on Z workload topology	Charles (AI Engineer) is a validated persona with no corresponding use case; as AI workloads on Z (MLz, Telum, Spyre) grow, understanding and managing the AI topology becomes a customer need	Future Opportunity: add a scenario to UC-05 (Application Discovery) for AI workload dependency mapping, or scope a standalone use case for 2027+ planning. Primary persona: Charles.
Change Governance and Traceability	Emerges repeatedly as a sub-theme in 3+ existing use cases but has no dedicated artifact	UC-14 added in this library. Requires H2 2027 ServiceNow integration for full realization.
Incident Investigation	Performance diagnosis (UC-11) covers performance incidents, but there is no use case for operational incidents — application outages, subsystem failures, unexpected restarts	Future Opportunity: add UC-15 or expand UC-11 to cover operational incident investigation. Concert for Z integration is the likely dependency.
Application Performance Testing	Performance testing as a proactive activity (not diagnosis) — before a go-live, before a change — is not explicitly captured as a scenario	Add scenario to UC-06 (Change Readiness): Pre-Go-Live Performance Baseline. Requires Atlas Test SKU (H1 2027).

Not Gaps — Correctly Scoped

The following were mentioned in source documents as potential use cases but are correctly classified as scenarios or supporting capabilities:

- **Application topology discovery** — a capability that enables UC-05, not a customer goal on its own
 - **PTF inventory query** — a step within UC-02 and UC-01, not a standalone use case
 - **Artifact generation** — a mechanism Atlas uses across all use cases, not a use case itself
 - **Agentic chat** — the interaction modality, not the customer goal
-

Recommended Library Size

14 core use cases is the recommended target, comprising the 13 defined above plus Storage Management (UC-15) once the Karla/Elsa persona work matures. This is at the upper bound of the 10–15 target range but justified because:

- IBM Z spans a genuinely broad domain (security, compliance, change, infrastructure, application, performance, DR)
- The personas reflect real organizational role separation — a storage admin and a compliance officer have meaningfully different goals
- Each use case maps to distinct customer conversations and distinct roadmap investments

If consolidation is needed to reach 10–12, the strongest candidates are:

Candidate for Consolidation	With	Rationale
UC-09 (Environment Parity and Drift Control)	UC-10 (Disaster Recovery Validation)	Both driven by cross-environment comparison; merge if the DR customer conversation is not sufficiently distinct

UC-13 (Regulatory Change Response)	UC-03 (Audit and Compliance)	High overlap in underlying Atlas data; split only if distinct buyer personas justify it
UC-06 (Change Readiness)	UC-02 (Patch Management)	Health check is often a step within patch management; merge if standalone health check use case is not prioritized commercially

The recommendation is to keep them split: the buyer personas and customer triggers are distinct even where the Atlas capabilities overlap.

Artifact Classification Reference

Use this table when encountering a new artifact and determining where it belongs.

If the artifact describes...	It is a...	Action
What business objective the customer is trying to accomplish	Use Case	Add or align to the use case index
A specific situation within that goal	Scenario	Link under the appropriate use case
A step-by-step conversation between a user and Atlas	Supporting artifact (conversation script)	Link from the scenario; do not elevate to use case
How Atlas transitions between states	Supporting artifact (state diagram / agent workflow)	Link from the use case; do not elevate to scenario

How the UI presents information	Supporting artifact (UX flow / screen flow)	Link from the scenario; do not elevate
A value domain Atlas delivers	Pillar	Reference in the pillar table; not a use case
An Atlas capability (topology graph, TIB, artifact generation)	Enabling capability	Reference in the use case "how it works"; not a use case

This document is the authoritative Atlas Use Case Library. For changes, contact Product Management. All use case artifacts, demo scenarios, and design documents should reference this taxonomy.